Java OOP — Part 3.5: Object Contracts and Value Design

Part 3.5 covers the methods every object inherits and how to design **value-like classes** well: correct `equals` and `hashCode`, ordering with `Comparable` and `Comparator`, immutable classes, and the modern building blocks (records, enums, sealed types). It continues from the overview of `Object` in 3.1.2 and the copying discussion in 3.3.7. (The pass-by-value rules, planned for this part, were already covered in 3.1.6.)

3.5.1 The Object Class Methods in Full

Every class inherits these methods from `java.lang.Object`. Some should be **overridden** to give your class meaningful behavior, some are `final`, and some should simply be avoided.

Method	Override?	Default behavior	Typical decision
`toString()`	**Yes**, almost always	`ClassName@hexHash`	Write a readable summary
`equals(Object)`	**Yes**, for value types	Same as `==`	Define logical equality (3.5.2)
`hashCode()`	**Yes**, whenever `equals` is overridden	Identity-based	Keep consistent with `equals` (3.5.2)
`getClass()`	No (`final`)	The runtime `Class` object	Use it, mostly inside `equals`
`clone()`	Rarely	Shallow copy, needs `Cloneable`	Prefer a copy constructor (3.3.7)
`finalize()`	**Never**	Does nothing	Deprecated for removal
`wait()`, `wait(long)`, `wait(long,int)`	No (`final`)	Releases the monitor and waits	Prefer `java.util.concurrent`
`notify()`, `notifyAll()`	No (`final`)	Wakes waiting threads	Same

toString()

`toString()` returns a **human-readable description** of the object. It is called implicitly by string concatenation, `println`, `String.valueOf`, loggers, debuggers and IDE watch windows.

```
class Point { int x, y; Point(int x, int y) { this.x = x; this.y = y; } }
System.out.println(new Point(1, 2));          // Point@1b6d3586   (default:
class name + @ + hex identity hash)

class Point2 {
    final int x, y;
    Point2(int x, int y) { this.x = x; this.y = y; }
    @Override public String toString() { return "Point2[x=" + x + ", y=" + y
+ "]"; }
}
System.out.println(new Point2(1, 2));         // Point2[x=1, y=2]
```

Guidelines

- Include the **significant state** in a compact, consistent format, and document it if other code may rely on it.
- **Never include secrets** (passwords, tokens, card numbers, personal data). `toString()` ends up in logs.
- **Avoid work that can fail or be slow:** do not trigger lazy loading (a JPA entity whose `toString` reads a lazy collection throws `LazyInitializationException` outside a session, or fires extra queries), and do not call remote services.
- **Beware cycles.** With bidirectional relationships (`Order` prints its lines, each line prints its `Order`) you get infinite recursion and a `StackOverflowError`. Print an id for the other side.
- Use `Objects.toString(value, "none")` for nullable parts, `String.format`, or `StringJoiner`. Lombok's `@ToString` and IDE generators work, but exclude sensitive and lazy fields.
- **Arrays** do not override `toString`: use `Arrays.toString(arr)` and `Arrays.deepToString(arr)`.
- **Records** and **enums** generate a good `toString` for you (3.5.5, 3.5.6).

getClass()

`getClass()` is `final` and returns the **runtime class** of the object, as a `Class<? extends X>` where `X` is the erasure of the static type.

```
Object o = new ArrayList<String>();
System.out.println(o.getClass().getName());         // java.util.ArrayList
System.out.println(o.getClass().getSimpleName());   // ArrayList
System.out.println(new int[0].getClass().getName());// [I
System.out.println(new Object(){}.getClass().getName());   // Main$1
(anonymous class)
```

- It tells you the **exact** type, while `instanceof` accepts subtypes. This difference matters in `equals` (3.5.2).
- Generics are erased: `new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()` is `true`.
- **Proxies change it.** Hibernate and Spring wrap objects in generated subclasses, so `entity.getClass()` may be `User$HibernateProxy$abc`, not `User`. An `equals` written with `getClass()` then fails between a proxy and the real object. For entities, use `instanceof` or the framework's unproxy helper.

clone() and finalize()

`clone()` is `protected`, makes a shallow field-by-field copy without calling a constructor, and needs the `Cloneable` marker (3.3.7). Prefer copy constructors. `finalize()` is deprecated for removal since Java 18; use `try-with-resources` and `Cleaner` (3.1.5).

wait(), notify() and notifyAll()

Every object can act as a **monitor**, the lock used by `synchronized`. These three methods let threads coordinate on a **condition** while holding that monitor.

```
class BoundedBuffer<T> {
    private final Queue<T> queue = new ArrayDeque<>();
    private final int capacity;
    BoundedBuffer(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) wait();       // release the lock
and sleep until notified
        queue.add(item);
        notifyAll();                                    // wake threads
waiting for "not empty"
    }
    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) wait();
        T item = queue.remove();
        notifyAll();                                    // wake threads
waiting for "not full"
        return item;
    }
}
```

Rules

- The calling thread must **own the object's monitor** (be inside `synchronized (obj)` or a synchronized method of that object). Otherwise the call throws `IllegalMonitorStateException`.
- `wait()` **releases the monitor** and puts the thread in the object's wait set. It re-acquires the monitor before returning.
- `notify()` wakes **one** waiting thread (which one is not specified); `notifyAll()` wakes **all** of them. A woken thread still has to get the lock, so it runs only after the notifier leaves its synchronized block.
- **Always call `wait()` in a loop that rechecks the condition.** A thread can wake up **spuriously**, and when several conditions share one monitor, a wake-up does not guarantee that *your* condition now holds. `if (cond) wait();` is a bug.
- Prefer `notifyAll()` unless you can prove that any single waiter can proceed; `notify()` can strand a thread that needed the signal.
- `wait(timeout)` returns after the timeout even without a notification.
- They are declared in `Object` because the lock belongs to the **object**, not the thread.

In modern code prefer `java.util.concurrent` (`BlockingQueue`, `Condition`, `CountDownLatch`, `Semaphore`), which are less error-prone. With virtual threads (Java 21), blocking in `synchronized` or `Object.wait()` **pinned** the carrier thread; Java 24 (JEP 491) removed that pinning for `synchronized`.

The Objects utility class

`java.util.Objects` holds null-safe helpers that make these methods easy to write.

Method	Use
`Objects.equals(a, b)`	Null-safe equality (true if both null)
`Objects.hash(a, b, c)`	Combined hash of several fields
`Objects.hashCode(o)`	Null-safe hash (0 for null)
`Objects.toString(o, "x")`	Null-safe string with a default
`Objects.requireNonNull(o, "msg")`	Fail fast with a clear message
`Objects.requireNonNullElse(o, d)`	Value or default
`Objects.compare(a, b, cmp)`	Null-tolerant comparison

Interview traps

- *What does the default `toString()` print?* `ClassName@` followed by the hex form of `hashCode()`.
- *Is the default `hashCode()` the memory address?* Not exactly. It is an identity hash generated by the JVM, stored in the object header once computed, and not guaranteed unique or tied to an address.
- *Why must `wait()` be in a loop?* Spurious wake-ups and shared conditions.
- *What if `notify()` is called without owning the monitor?* `IllegalMonitorStateException`.
- *Why are `wait` and `notify` in `Object` and not in `Thread`?* Because any object can serve as a lock, and the lock is a property of the object.
- *Why can `getClass()` cause trouble in `equals`?* Frameworks subclass objects with proxies, so exact class comparison fails.

3.5.2 equals() and hashCode(): the Contract

Two questions every Java developer must answer: *when are two objects "the same"*, and *how do hash-based collections find them?* `equals()` answers the first and `hashCode()` supports the second, and they must agree.

Identity vs equality

```
String a = new String("hi");
String b = new String("hi");
System.out.println(a == b);          // false: identity (two different
objects)
System.out.println(a.equals(b));     // true:  logical equality (same
characters)
```

`Object.equals` is just `==`. For **value types** (money, point, email address, order line) you override `equals` so that objects with the same state count as equal. Many APIs depend on it: `List.contains/indexOf/remove(Object)`, `HashSet`, `HashMap` keys, `assertEquals`, `Set.copyOf`, and so on.

The equals contract

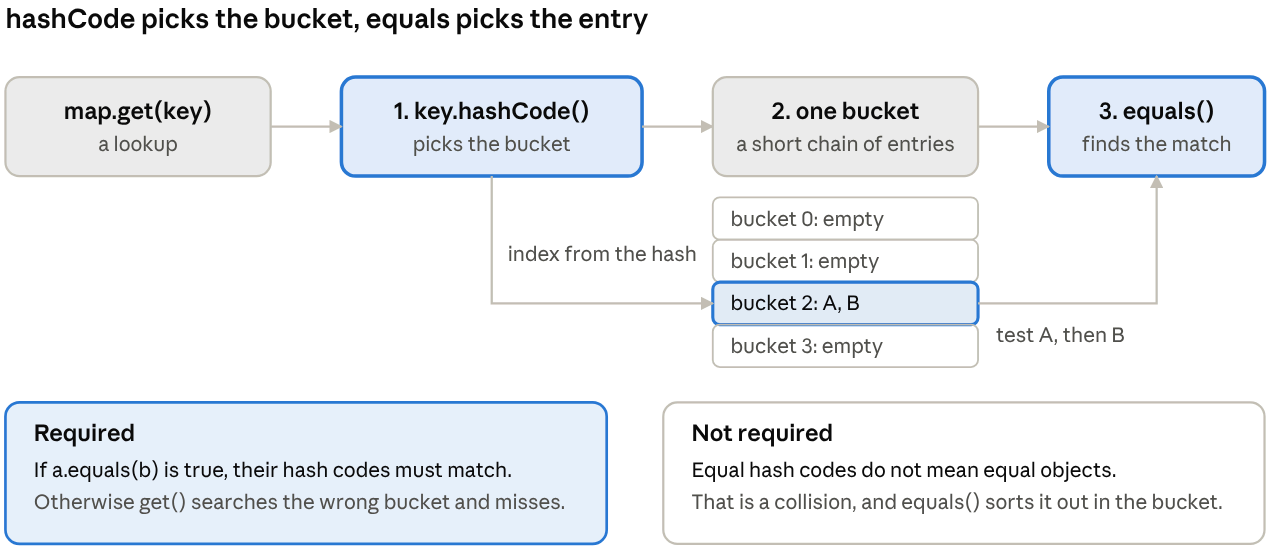
From `Object.equals`'s documentation, for non-null references `x`, `y`, `z`:

Property	Rule	Typical way to break it
Reflexive	`x.equals(x)` is `true`	Rare; a faulty comparison of a field
Symmetric	`x.equals(y)` iff `y.equals(x)`	Comparing different classes, such as a `String` with a case-insensitive wrapper
Transitive	`x.equals(y)` and `y.equals(z)` imply `x.equals(z)`	Subclass that adds a field (below)
Consistent	Repeated calls give the same answer while nothing changed	Using volatile data (time, random, a remote call)
Non-null	`x.equals(null)` is `false`	Not checking for `null` (throws instead)

The hashCode contract

1. **Within one run**, `hashCode()` returns the same value as long as the information used by `equals` is unchanged.
2. **If `x.equals(y)`, then `x.hashCode() == y.hashCode()`.** This is the rule that matters.
3. If `x` and `y` are **not** equal, their hash codes are **not required** to differ; but different values for unequal objects improve the performance of hash tables.

How hash-based collections use them



How a hash lookup uses hashCode and equals

A `HashMap.get(key)` first calls `key.hashCode()` to find the **bucket**, then walks the entries in that bucket and uses `equals()` to find the right one. Break rule 2 and an equal key is looked up in a **different bucket**, so the entry is never found.

```
class Money {
    final long paise;
    Money(long paise) { this.paise = paise; }
    @Override public boolean equals(Object o) {                 //
overridden...
        return o instanceof Money m && m.paise == paise;
    }
    // hashCode NOT overridden: still identity-based
}

Set<Money> set = new HashSet<>();
set.add(new Money(500));
System.out.println(set.contains(new Money(500)));   // false (usually): equal
objects, different hash codes
System.out.println(set.size());                      // add(new Money(500))
again would make size 2: a "set" with duplicates
```

Rule: whenever you override `equals`, override `hashCode` in the same change.

Writing them correctly

```
public final class Point {
    private final int x, y;
    private final double weight;
    private final String label;                          // may be null

    // constructor omitted

    @Override public boolean equals(Object o) {
        if (this == o) return true;                      // 1. same reference:
shortcut
        if (!(o instanceof Point p)) return false;       // 2. null or wrong
type: false (instanceof is false for null)
        return x == p.x                                  // 3. compare each
significant field
            && y == p.y
            && Double.compare(weight, p.weight) == 0     //    floats and
doubles: Double.compare
            && Objects.equals(label, p.label);           //    references:
Objects.equals (null-safe)
    }
```

```
    @Override public int hashCode() {
        return Objects.hash(x, y, weight, label);      // the SAME fields, in
any fixed order
    }
}
```

Recipe

1. Shortcut with `this == o`.
2. Check the type: `instanceof` (or `getClass()`, see below). A `null` argument must return `false`, not throw.
3. Compare the **significant fields**: primitives with `==`; `float` and `double` with `Float.compare` and `Double.compare` (they handle `NaN` and `-0.0` consistently with `Double.equals`); references with `Objects.equals`; arrays with `Arrays.equals` (or `deepEquals`).
4. In `hashCode`, use **exactly the fields used by `equals`** (a subset is legal, extras are a bug). `Objects.hash(...)` is concise; on hot paths use the manual form `31 * result + field` to avoid creating an array and boxing.
5. Make the fields `final` (or the whole class immutable) so that the hash code cannot change after the object is stored in a hash-based collection.

```
@Override public int hashCode() {                 // manual form, same idea
as String and List
    int h = Integer.hashCode(x);
    h = 31 * h + Integer.hashCode(y);
    h = 31 * h + Double.hashCode(weight);
    h = 31 * h + Objects.hashCode(label);
    return h;
}
```

`31` is an odd prime that the JIT turns into a shift and subtraction (`31 * h == (h << 5) - h`); using a prime spreads values and avoids systematic collisions.

instanceof or getClass()?

	`instanceof`	`getClass() == o.getClass()`
Subclass instance equal to a parent instance	Yes, if state is equal	No: exact classes must match

	`instanceof`	`getClass() == o.getClass()`
Satisfies Liskov substitution	**Yes**	**No** (an anonymous or proxy subclass is "not equal")
Symmetry when a subclass adds state	**Breaks**	Preserved
Works with Hibernate and Spring proxies	Yes	**No**

The unsolvable problem: extending a class and adding a value component.

```
class Point {
    final int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
    @Override public boolean equals(Object o) { return o instanceof Point p
&& p.x == x && p.y == y; }
}
class ColorPoint extends Point {
    final String color;
    ColorPoint(int x, int y, String color) { super(x, y); this.color = color;
}
    @Override public boolean equals(Object o) {
        return o instanceof ColorPoint c && super.equals(c) &&
c.color.equals(color);
    }
}
Point p = new Point(1, 2);
ColorPoint cp = new ColorPoint(1, 2, "red");
System.out.println(p.equals(cp));      // true   ColorPoint IS-A Point with
the same x and y
System.out.println(cp.equals(p));      // false  p is not a ColorPoint:
SYMMETRY BROKEN
```

If `ColorPoint.equals` instead ignores the color when comparing with a plain `Point`, symmetry holds but **transitivity** breaks (`red.equals(p)`, `p.equals(blue)`, yet `red.equals(blue)` is false). There is no way to fix this inside the hierarchy. Choose one: make the classes `final`, use **composition** (a `ColorPoint` *has a* `Point`), use **records**, or use `getClass()` and give up substitutability.

Common bugs

- **Overriding `equals` but not `hashCode`** (above): duplicates in sets and failed lookups.
- **Overloading instead of overriding:** `equals(Point p)` does not override `equals(Object)`. Always use `@Override` (3.3.4).
- **Different fields in the two methods:** `equals` uses `id` and `name`, `hashCode` only `id` is legal; the other way round is a bug.
- **Mutable fields in a key.** If a field used by `hashCode` changes after insertion, the object sits in the wrong bucket and is effectively lost:

```
Set<Point2> set = new HashSet<>();
Point2 p = new Point2(1, 2);           // a mutable class with equals and
hashCode on x and y
set.add(p);
p.x = 99;                              // hash changes
System.out.println(set.contains(p));   // false: looked up in the new bucket,
stored in the old
```

- **A constant `hashCode`** (`return 1;`) satisfies the contract but turns every lookup into a linear scan (Java 8 can treeify long chains, but only helps if the key is `Comparable`).
- **Comparing `String`s or boxed values with `==`** inside `equals`.
- **Using non-deterministic data** (current time, random values, database state) in `equals`.

Where each collection looks

Collection	Uses
`HashMap`, `HashSet`, `LinkedHashMap`, `ConcurrentHashMap`	`hashCode()` then `equals()`
`ArrayList.contains/indexOf/remove(Object)`, `LinkedList`	`equals()` only
`TreeMap`, `TreeSet`	`compareTo` / `Comparator` only (3.5.3), **not** `equals`
`Arrays.asList(...).contains`, `Objects.equals`	`equals()`

Special cases

- **Records** generate `equals`, `hashCode` and `toString` from **all components**. A component that is an **array** is compared by reference, a classic trap (3.5.5).
- **Enums:** `equals` is final and identity-based; `==` is the idiom.
- **JPA entities:** a database-generated `id` is `null` until the entity is saved, so an `equals` or `hashCode` based on `id` changes after `persist()`. Common safe choices: base them on a natural **business key** that never changes, or use an `id`-based `equals` with a **constant** `hashCode` per entity class (accepting slower hashing). Do not base them on lazy collections. Lombok's `@EqualsAndHashCode` on entities needs care for the same reasons.
- **`Arrays`:** `a.equals(b)` on arrays is identity; use `Arrays.equals` and `Arrays.hashCode`.

Hash quality

`HashMap` spreads the bits of `hashCode` (`h ^ (h >>> 16)`) and masks by table size. A **good** hash code gives an even spread, so buckets stay short and lookups stay near O(1). A poor one (many collisions) degrades toward O(n), or O(log n) once a bucket is treeified and the key is `Comparable`. Hash flooding attacks exploit predictable hashes, which is why modern JDK collections treeify.

Interview traps

- *What is the relationship between `equals` and `hashCode`?* Equal objects must have equal hash codes; the converse is not required.
- *What happens if you override only `equals`?* Hash-based collections break: equal objects land in different buckets, duplicates appear, lookups fail.
- *Is it legal for `hashCode` to return a constant?* Yes, but it destroys performance.
- *Can unequal objects have the same hash code?* Yes (a collision).
- *Why not use mutable fields in `hashCode`?* The hash can change while the object is stored, losing it.
- *Why is `equals(Point)` not an override?* The parameter type must be `Object`.
- *`instanceof` or `getClass()`?* `instanceof` with a `final` class or record is the safest; `getClass()` keeps symmetry for non-final classes but breaks substitutability and proxies.

3.5.3 Comparable vs Comparator

Sorting, `TreeMap`, `TreeSet`, `PriorityQueue`, binary search and `min`/`max` all need an **ordering**. Java offers two ways to supply one: the class itself defines a **natural order** (`Comparable`), or an outside object defines **an** order (`Comparator`).

	`Comparable<T>`	`Comparator<T>`
Package	`java.lang`	`java.util`
Method	`int compareTo(T other)`	`int compare(T a, T b)`
Where the ordering lives	**Inside** the class being sorted	**Outside**, in a separate object or lambda
Number of orderings	**One** (the "natural" order)	**Any number**
Needs to change the class?	Yes	No (works for classes you do not own)
Typical use	Value types with an obvious order: `String`, `Integer`, `LocalDate`	Sort by different criteria, or for third-party types
Lambda friendly	No (a class implements it)	**Yes** (functional interface)

Both return a **negative** number if the first is smaller, **zero** if equal in order, and **positive** if greater.

Comparable: natural ordering

```
public class Employee implements Comparable<Employee> {
    private final String name;
    private final int salary;
    Employee(String name, int salary) { this.name = name; this.salary =
salary; }

    @Override public int compareTo(Employee other) {
        return Integer.compare(this.salary, other.salary);          //
natural order: by salary, ascending
    }
}

List<Employee> staff = new ArrayList<>(List.of(new Employee("Ravi", 70), new
Employee("Asha", 50)));
Collections.sort(staff);            // uses compareTo
staff.sort(null);                   // same: a null comparator means natural
order
TreeSet<Employee> set = new TreeSet<>(staff);   // ordered by compareTo
```

JDK classes with a natural order: `String` (lexicographic by UTF-16 units), the numeric wrappers, `Character`, `Boolean`, `BigInteger`, `BigDecimal`, `LocalDate` and the other `java.time` types, and enums (by declaration order).

The compareTo contract (for non-null `x`, `y`, `z`):

1. **Antisymmetry:** `sgn(x.compareTo(y)) == -sgn(y.compareTo(x))` (and one throws iff the other does).
2. **Transitivity:** `x > y` and `y > z` imply `x > z`.
3. **Consistency of equality in order:** if `x.compareTo(y) == 0`, then `x` and `y` must compare the same way against every other `z`.
4. `x.compareTo(null)` should throw `NullPointerException` (unlike `equals`).
5. **Strongly recommended:** be **consistent with `equals`**, that is, `x.compareTo(y) == 0` exactly when `x.equals(y)`.

Why consistency with equals matters

`HashSet` uses `equals`/`hashCode`, but `TreeSet` and `TreeMap` use **only `compareTo`**. If the two disagree, the same data behaves differently in different collections.

```
BigDecimal a = new BigDecimal("1.0");
BigDecimal b = new BigDecimal("1.00");
System.out.println(a.equals(b));                        // false (scale
differs)
System.out.println(a.compareTo(b));                     // 0     (numerically
equal)
System.out.println(new HashSet<>(List.of(a, b)).size());   // 2
System.out.println(new TreeSet<>(List.of(a, b)).size());   // 1  (b is
treated as a duplicate and dropped)
```

The same trap appears with a `Comparator` that looks at only some fields:

```
TreeSet<Employee> byAge = new TreeSet<>
(Comparator.comparingInt(Employee::age));
byAge.add(new Employee("Asha", 30));
byAge.add(new Employee("Ravi", 30));      // compare == 0 against Asha:
treated as a duplicate and NOT added
System.out.println(byAge.size());          // 1 : data silently lost
```

Fix: add **tie-breakers** so that distinct objects never compare as 0 (`.thenComparing(Employee::name)`).

Comparator: custom orderings

In the examples below, assume `Employee` is a record with the accessors `name()`, `department()`, `salary()`, `age()` and `bonus()` (a nullable `Integer`).

```
Comparator<Employee> bySalary = (a, b) -> Integer.compare(a.salary(),
b.salary());
staff.sort(bySalary);

// Anonymous class form (older style)
staff.sort(new Comparator<Employee>() {
    @Override public int compare(Employee a, Employee b) { return
a.name().compareTo(b.name()); }
});
```

Build comparators with the factory methods (Java 8+). They are shorter and avoid arithmetic mistakes:

```
Comparator<Employee> c =
    Comparator.comparing(Employee::department)                // by
department (needs Comparable keys)
              .thenComparingInt(Employee::salary)              // then by
salary (primitive: no boxing)
              .reversed()                                      // reverse the
whole thing
              .thenComparing(Employee::name);                  // then by
name ascending

staff.sort(Comparator.comparing(Employee::name,
String.CASE_INSENSITIVE_ORDER));   // custom key comparator
staff.sort(Comparator.comparing(Employee::bonus,
Comparator.nullsLast(Comparator.naturalOrder())));   // nulls last
Map.Entry<String, Integer> top = Collections.max(map.entrySet(),
Map.Entry.comparingByValue());
```

Method	Purpose
`Comparator.comparing(keyExtractor)`	Order by a key that is `Comparable`
`comparingInt`, `comparingLong`, `comparingDouble`	Same, without boxing
`thenComparing(...)`, `thenComparingInt(...)`	Tie-breakers
`reversed()`	Reverse this comparator
`Comparator.naturalOrder()`, `reverseOrder()`	Natural order and its reverse
`Comparator.nullsFirst(c)`, `nullsLast(c)`	Wrap to allow `null` elements
`Map.Entry.comparingByKey()`, `comparingByValue()`	For map entries
`String.CASE_INSENSITIVE_ORDER`	Ignore case (for locale-aware text use `java.text.Collator`)

One inference trap: `Comparator.comparing(e -> e.salary()).reversed()` may fail to compile because the lambda's type is not inferred before `.reversed()`; use `Comparator.comparing(Employee::salary).reversed()` or write the type: `(Employee`

`e) -> e.salary()`.

Using orderings

```
list.sort(cmp);                              Collections.sort(list, cmp);
Arrays.sort(array, cmp);                     Arrays.sort(intArray);        //
primitives: natural order only
new TreeSet<>(cmp);   new TreeMap<>(cmp);   new PriorityQueue<>(cmp);
stream.sorted(cmp);   stream.max(cmp);      stream.min(cmp);
Collections.max(coll, cmp);
Collections.binarySearch(sortedList, key, cmp);
```

How the JDK sorts. `List.sort` and `Arrays.sort` on objects use **TimSort**, a stable merge-based sort (O(n log n), nearly O(n) on partly sorted data). **Stable** means elements that compare equal keep their original relative order, which is what makes multi-pass sorting (sort by name, then by department) work. `Arrays.sort` on primitives uses a dual-pivot quicksort (not stable, which is irrelevant for primitives).

Classic comparator bugs

1. Subtraction overflow.

```
(a, b) -> a.age() - b.age()          // WRONG for large or negative values
Integer.compare(a.age(), b.age())    // RIGHT
// Example: a.age() = Integer.MAX_VALUE, b.age() = -1: the difference
overflows to a negative number, so the order is reversed
```

2. Comparing floating-point with `<` and `>`. `NaN` is neither smaller nor greater than anything, so such a comparator violates transitivity. Use `Double.compare`.

3. An inconsistent comparator (random results, comparing mutable state that changes during the sort, ignoring `null`) can make TimSort throw `IllegalArgumentException: Comparison method violates its general contract!`, but only for some inputs, which makes it hard to reproduce.

4. Returning only `1` or `0` (`return a.x > b.x ? 1 : 0;`) is not antisymmetric.

5. `compareTo` fields that differ from `equals`. See the `TreeSet` example above.

6. Mutating a sort key while the object sits in a `TreeSet` or `TreeMap` corrupts the tree's order; the element becomes unfindable.

Choosing between them

- Give a class **`Comparable`** when it has one obvious, universally expected order (dates, versions, money in one currency). Keep it consistent with `equals`.
- Use a **`Comparator`** for every *other* order, for classes you do not own, and when the order depends on context (sort a table by the column the user clicked).
- A class may do both: a natural order by `id` and several `Comparator` constants for other views.

```
public record Version(int major, int minor, int patch) implements
Comparable<Version> {
    private static final Comparator<Version> ORDER =

Comparator.comparingInt(Version::major).thenComparingInt(Version::minor).then
ComparingInt(Version::patch);
    @Override public int compareTo(Version o) { return ORDER.compare(this,
o); }
}
```

Interview traps

- *Comparable vs Comparator?* Natural single order inside the class (`compareTo`) vs any number of external orders (`compare`).
- *Why not `a.x - b.x` in `compare`?* Integer overflow reverses the sign for large differences; use `Integer.compare`.
- *What does "consistent with equals" mean, and what breaks otherwise?* `compareTo == 0` iff `equals` ; `TreeSet` / `TreeMap` use `compareTo` for identity, so data can be silently dropped, and `HashSet` and `TreeSet` disagree.
- *Can a `TreeSet` hold `null`?* Not with natural ordering (`compareTo(null)` throws); a custom comparator that handles `null` can.
- *Is `Collections.sort` stable?* Yes (TimSort).
- *Does `Arrays.sort(int[])` accept a `Comparator`?* No; use `Integer[]` , or sort a primitive array naturally and reverse it.

3.5.4 Immutability: Designing an Immutable Class

An **immutable** object cannot change its state after it is constructed. Every "modification" produces a **new** object. Familiar immutable types: `String`, the wrapper classes (`Integer`, `Double`), `BigInteger`, `BigDecimal`, `LocalDate` and the rest of `java.time`, `Optional`, `UUID`, and the collections made by `List.of`, `Set.of`, `Map.of` and `List.copyOf`.

Why immutability is valuable

Benefit	Explanation
Thread safety without locks	Nothing can change, so no data races. Shared freely across threads
Simple reasoning	An object is valid once, at construction, and stays valid. No "who changed this?" bugs
Safe sharing and caching	No defensive copies needed; instances can be reused (`Integer.valueOf`, the String pool)
Reliable map keys and set elements	The hash code never changes (3.5.2)
Failure atomicity	An operation that throws leaves no half-modified object behind
Memory-model guarantee	`final` fields are safely visible to other threads once the constructor ends (3.2.2)

Costs: each change allocates a new object, which can be expensive for large objects that change often (use a mutable builder for assembly, like `StringBuilder`), and some frameworks (JPA entities, classic JavaBeans binding) want mutable classes with setters.

The recipe for an immutable class

1. **No mutator methods** (no setters, no methods that modify state).
2. **Make the class `final`** (or give it only `private` constructors and offer static factories), so a subclass cannot add mutability.
3. **Make all fields `final`.**
4. **Make all fields `private`.**

5. **Guarantee exclusive access to mutable components.** If a field refers to a mutable object (an array, a `Date`, an `ArrayList`), make a **defensive copy** when you receive it (constructor) and never hand out the internal reference (accessors).
6. **Do not let `this` escape** the constructor, and validate in the constructor (fail fast).

A class that only looks immutable

```
public final class BadOrder {                          // final class, final
field, no setters... but still mutable
    private final List<String> items;
    public BadOrder(List<String> items) { this.items = items; }      //
stores the caller's list
    public List<String> getItems() { return items; }                 // hands
out the internal list
}

List<String> mine = new ArrayList<>(List.of("pen"));
BadOrder order = new BadOrder(mine);
mine.add("laptop");                    // the caller changes the order from
outside
order.getItems().clear();              // and so does anyone holding the
order
```

`final` stops reassigning the field, not changing the list it refers to (3.2.2). Rule 5 is what is broken here.

The correct version

```
public final class Order {
    private final long id;
    private final Instant created;                  // Instant is immutable:
safe to share
    private final List<String> items;               // will hold an
immutable list

    public Order(long id, Instant created, List<String> items) {
        this.id = id;
        this.created = Objects.requireNonNull(created, "created");
        this.items = List.copyOf(items);            // defensive copy IN;
also immutable; rejects nulls
    }
```

```
    public long id()              { return id; }
    public Instant created()      { return created; }
    public List<String> items()   { return items; }     // safe to return: it
is already immutable

    public Order withItem(String item) {                // "wither": returns
a NEW object
        List<String> next = new ArrayList<>(items);
        next.add(item);
        return new Order(id, created, next);
    }
}
```

Now neither the original caller's list nor the accessor can alter the order, and `order.items().add("x")` throws `UnsupportedOperationException`.

Defensive copies for older mutable types

```
public final class Booking {
    private final Date start;                           // java.util.Date is
mutable (prefer java.time.Instant)
    private final int[] seats;

    public Booking(Date start, int[] seats) {
        this.start = new Date(start.getTime());         // copy in
        this.seats = seats.clone();                     // copy in (shallow
copy is enough for int[])
    }
    public Date start()   { return new Date(start.getTime()); }   // copy out
    public int[] seats()  { return seats.clone(); }               // copy out
}
```

Make the copy **before** validating, and validate the **copy**, otherwise another thread can change the argument between the check and the copy (a time-of-check/time-of-use gap).

Immutable vs unmodifiable vs final

Term	Meaning	Example
`final` reference	The variable cannot be reassigned	`final List<String> l = new ArrayList<>();` still mutable
Unmodifiable view	A read-only wrapper; the **underlying collection can still change**	`Collections.unmodifiableList(list)`
Immutable	Nobody can change it, ever	`List.of(...)`, `List.copyOf(...)`, `String`

```
List<String> base = new ArrayList<>(List.of("a"));
List<String> view = Collections.unmodifiableList(base);
List<String> snap = List.copyOf(base);
base.add("b");
System.out.println(view);    // [a, b]   the view follows the original
System.out.println(snap);    // [a]      the snapshot does not
```

Shallow vs deep immutability. `List.copyOf` makes the **list** immutable, but if its elements are mutable (`List<StringBuilder>`, `List<Person>` with setters), they can still change. A class is **deeply immutable** only if everything reachable from it is immutable, or hidden behind defensive copies.

Producing changed copies

Three common styles:

- **Withers:** `order.withItem("pen")` returns a new `Order`, as `LocalDate.plusDays(1)` does.
- **Builder:** assemble with a mutable helper, then build the immutable object (3.2.4's `Pizza.Builder`).
- **Static factories** that can **cache** instances, for common values: `Integer.valueOf(5)`, `Boolean.valueOf(true)`, `BigInteger.ZERO`.

```
LocalDate d1 = LocalDate.of(2026, 1, 31);
LocalDate d2 = d1.plusMonths(1);          // new object; d1 is unchanged
String s = "abc";
String t = s.toUpperCase();               // new String; s is unchanged
```

Lazy and cached fields

An immutable class may still compute and cache a value (`String` caches `hashCode` in a non-final field). That is acceptable when the cached value is **derived deterministically** from the final state, the field is only an optimization, and a benign data race cannot produce a wrong answer.

```
public final class Polygon {
    private final List<Point> points;
    private int hash;                                   // 0 = not computed yet
(benign race: always the same value)
    @Override public int hashCode() {
        int h = hash;
        if (h == 0) { h = points.hashCode(); hash = h; }
        return h;
    }
}
```

Immutability in real projects

- **Value objects and DTOs** (money, ids, addresses, API request and response bodies) should be immutable: use **records** (3.5.5).
- **JPA entities** usually cannot be (the provider needs a no-arg constructor and mutates fields), but you can mark read-only data with Hibernate's `@Immutable` and expose immutable DTOs outward.
- **Configuration and domain events** are naturally immutable.
- Libraries such as Lombok's `@Value`, Immutables and AutoValue generate the boilerplate; records now cover most cases.
- **Serialization can break immutability:** a deserializer may bypass your constructor. Records are safe (they deserialize through the canonical constructor); for other classes use `readResolve` or avoid Java serialization.

Converting a mutable class to an immutable one: checklist

1. Remove setters; replace changes with withers or a builder.
2. Make the class `final` and all fields `private final`.
3. Copy mutable inputs in the constructor and replace mutable outputs with copies or immutable views.

4. Replace `Date` with `Instant` / `LocalDate`, arrays and lists with immutable collections where you can.
5. Validate in the constructor; do not leak `this`.
6. Ensure `equals`, `hashCode` and `toString` use the final fields (3.5.2).

Interview traps

- *How do you make a class immutable?* Final class, private final fields, no setters, defensive copies of mutable components, no `this` escape.
- *Is a class with only `final` fields immutable?* Not if a field refers to a mutable object that is shared or exposed.
- *`final List<String>` vs immutable list?* `final` only fixes the reference; `List.of` or `List.copyOf` makes the content immutable.
- *`Collections.unmodifiableList` vs `List.copyOf`?* A read-only view of the original vs an independent immutable snapshot.
- *Why is `String` immutable?* Security, thread safety, cached hash code, pooling.
- *Does immutability cost performance?* It allocates more objects, but removes locking and defensive copying, and short-lived garbage is cheap in modern collectors.

3.5.5 Records: Immutable Data Carriers

A **record** (standard since Java 16) is a concise way to declare a class whose only job is to **carry data**. You list the **components**, and the compiler writes the boilerplate that 3.5.2 and 3.5.4 asked you to write by hand.

```
public record Point(int x, int y) { }
```

That one line gives you a class with:

- a `private final` field per component (`x`, `y`);
- a **canonical constructor** `Point(int x, int y)`;
- **accessor methods** named after the components: `x()` and `y()` (not `getX()`);
- **`equals`** and **`hashCode`** based on all components;
- a **`toString`** such as `Point[x=1, y=2]`;
- the class is implicitly **`final`** and extends `java.lang.Record`.

```
Point p = new Point(3, 4);
System.out.println(p.x() + "," + p.y());                  // 3,4
System.out.println(p);                                      // Point[x=3,
y=4]
System.out.println(p.equals(new Point(3, 4)));              // true
System.out.println(new HashSet<>(List.of(p, new Point(3, 4))).size());   // 1
```

Validation and normalization: the compact constructor

A **compact canonical constructor** has no parameter list. It runs at the start of construction, can validate and **reassign the parameters**, and the fields are then assigned automatically at the end.

```
public record Range(int lo, int hi) {
    public Range {                                          // compact
constructor
        if (lo > hi) throw new IllegalArgumentException("lo " + lo + " > hi "
+ hi);
    }
}

public record Email(String value) {
    public Email {
        Objects.requireNonNull(value, "value");
        value = value.strip().toLowerCase();                // normalize:
assigns the PARAMETER, not the field
        if (!value.contains("@")) throw new IllegalArgumentException("not an
email: " + value);
    }
}
System.out.println(new Email("  Asha@Example.COM ").value());   //
asha@example.com
```

You may also write the **full canonical constructor** with the explicit parameter list, in which case you must assign every field yourself. Both forms must be at least as accessible as the record.

What a record may and may not contain

Allowed	Not allowed
Components, constructors (compact or canonical)	**Instance fields** beyond the components
Other constructors that delegate with `this(...)`	Extending another class (the parent is always `Record`)
Instance methods, including overriding accessors, `equals`, `hashCode`, `toString`	Being extended (records are `final`)
`static` fields, methods, initializers, nested types	Being `abstract`
Implementing **interfaces**	Setters, or assigning a field after construction
Generic type parameters, annotations	Declaring `native` methods
Being nested, local, or top-level	

```
public record Money(long paise, String currency) implements Comparable<Money>
{
    public static final Money ZERO_INR = new Money(0, "INR");           //
static member

    public Money(long paise) { this(paise, "INR"); }                    //
extra constructor must delegate

    public static Money ofRupees(long rupees) { return new Money(rupees *
100, "INR"); }   // static factory

    public Money plus(Money other) {                                    //
behavior: returns a NEW record
        if (!currency.equals(other.currency)) throw new
IllegalArgumentException("currency mismatch");
        return new Money(paise + other.paise, currency);
    }
    @Override public int compareTo(Money o) { return Long.compare(paise,
o.paise); }
}
```

Records are shallowly immutable

The component **references** can never change, but a mutable object they point to still can. Apply the immutability recipe (3.5.4) to mutable components:

```
public record Team(String name, List<String> members) {
    public Team {
        members = List.copyOf(members);          // defensive copy in;
immutable list; accessor is then safe
    }
}
```

The array trap. An array component is compared **by reference**, printed as `[I@...`, and exposed through the accessor:

```
record Scores(int[] values) { }
Scores a = new Scores(new int[]{1, 2});
Scores b = new Scores(new int[]{1, 2});
System.out.println(a.equals(b));         // false: arrays compare by
reference
System.out.println(a);                   // Scores[values=[I@3a4afd8d]

// Fix: use List<Integer> or override equals, hashCode, toString with
Arrays.* and copy in the constructor and accessor.
```

Customizing the generated members

You may override the **accessors**, `equals`, `hashCode` and `toString`. An accessor must be `public`, have the same name and return type, and no parameters.

```
record Person(String name, int age) {
    @Override public String name() { return name.toUpperCase(); }          //
custom accessor
    @Override public String toString() { return name + " (" + age + ")"; } //
custom text
}
```

If you override `equals`, keep it consistent with `hashCode` (3.5.2). In most cases do not override them: the generated versions compare **all components**, `primitives` by value and references with `Objects.equals`.

Records with generics, interfaces and nesting

```
record Pair<A, B>(A first, B second) { }                    // a typed tuple
Pair<String, Integer> p = new Pair<>("age", 30);

sealed interface Shape permits Circle, Rect { }              // records fit
sealed hierarchies perfectly
record Circle(double r) implements Shape { }
record Rect(double w, double h) implements Shape { }

static Stream<Order> topOrders(List<Order> orders) {
    record Scored(Order order, double score) { }             // a LOCAL
record: a throwaway tuple inside a method
    return orders.stream().map(o -> new Scored(o, score(o)))

.sorted(Comparator.comparingDouble(Scored::score).reversed())
                .map(Scored::order);
}
```

Records and pattern matching (Java 21)

Because a record's components are **transparent**, it can be **deconstructed**:

```
Object o = new Point(3, 4);
if (o instanceof Point(int x, int y)) System.out.println(x + y);      // 7

record Line(Point from, Point to) { }
static double length(Object o) {
    if (o instanceof Line(Point(var x1, var y1), Point(var x2, var y2))) {
// nested record pattern
        return Math.hypot(x2 - x1, y2 - y1);
    }
    return 0;
}

static String describe(Shape s) {
    return switch (s) {
        case Circle(double r)       -> "circle " + r;
        case Rect(double w, double h) -> "rect " + w + "x" + h;
    };
}
```

This is the other half of the record story: records **construct** data neatly and pattern matching **takes it apart** neatly.

Where to use records

- **DTOs and API models:** request and response bodies (Spring MVC and Jackson 2.12+ support records), event messages, query results and Spring Data projections.
- **Value objects:** `Money`, `Email`, `OrderId`, `Coordinates`; good map keys, since `equals` and `hashCode` are free and the object is immutable.
- **Multiple return values** and temporary tuples (`record MinMax(int min, int max)`).
- **Members of sealed hierarchies** (algebraic data types).

Where not to use them

- **JPA entities:** the provider needs a no-arg constructor and mutable fields. (Hibernate 6.2 and later accept records as `@Embeddable` types.)
- Classes with **behavior-heavy, mutable state** or **inheritance**.
- Types that should hide their representation: a record's components are its public API, and renaming a component is a breaking change.

Records vs classes vs Lombok

	Hand-written class	Lombok `@Value`	Record
Boilerplate	Large	Generated by annotation processing	**Built into the language**
Accessor naming	`getX()`	`getX()`	`x()`
Extra instance fields	Yes	Yes	**No**
Can extend a class	Yes	Yes	**No**
Works with pattern matching deconstruction	No	No	**Yes**
Tooling dependency	None	Lombok plugin	None

Interview traps

- *Are records immutable?* Shallowly: component references are fixed, but a mutable component object can still change.
- *What does the compiler generate for a record?* Fields, canonical constructor, accessors, `equals`, `hashCode`, `toString`.
- *Can a record extend a class or be extended?* No on both counts; it can implement interfaces.
- *Can a record have instance fields or setters?* No.
- *What is a compact constructor?* A canonical constructor without a parameter list, used to validate or normalize parameters; field assignment happens automatically afterwards.
- *Why can an array component be a problem?* `equals`, `hashCode` and `toString` treat arrays by reference.
- *Accessor names?* The component name, `x()`, not `getX()`.

3.5.6 Enums: Full Classes with a Fixed Set of Constants

An **enum** declares a type whose values are a **fixed, named set of constants**. In Java an enum is a **full class**: each constant is a single, shared object that can carry fields and behavior.

```
public enum Status { NEW, PAID, SHIPPED, CANCELLED }

Status s = Status.PAID;
System.out.println(s);                  // PAID
System.out.println(s == Status.PAID);  // true: constants are singletons, so
== is correct
// Status x = new Status();            // ERROR: enums cannot be instantiated
// Status y = "PAID";                  // ERROR: type safety: only Status
constants are accepted
```

Why not `static final int` constants?

Before Java 5, programmers wrote `public static final int NEW = 0, PAID = 1;`. That "int enum pattern" has real problems that enums fix:

Problem with int constants	Enum solution
No type safety: any `int` is accepted, `setStatus(42)` compiles	Parameter type is `Status`: only its constants fit
No namespace: `NEW` may clash with other constants	`Status.NEW` is qualified
Brittle: values are inlined into callers (3.2.1), so renumbering needs a full recompile	Constants are objects referenced at run time
Not printable: logs show `1`	`toString()` gives `PAID`
No behavior: you need `switch` tables everywhere	Fields and methods live on the constant

What an enum really is

- It implicitly **extends `java.lang.Enum<E>`**, so it **cannot extend another class**, but it **can implement interfaces**.
- Each constant is a `public static final` instance of the enum class, created **once**, when the class is initialized.
- The constructor is implicitly **`private`**; you cannot call `new`.
- The class is implicitly `final` (unless a constant has its own body, see below).
- It works in `switch`, as a `Map` key, and is automatically `Comparable` and `Serializable`.

Methods every enum has

Method	Meaning
`values()`	A **new array** of all constants, in declaration order
`valueOf(String)`	Constant with that exact name; throws `IllegalArgumentException` if none
`name()`	The constant's declared name (final)
`toString()`	Same as `name()` by default; can be overridden for display
`ordinal()`	Zero-based position in the declaration

Method	Meaning
`compareTo(E)`	Compares by `ordinal()` (final)
`equals`, `hashCode`	Identity-based and `final`
`getDeclaringClass()`	The enum type (useful with constant bodies)

```
for (Status st : Status.values()) System.out.println(st.ordinal() + " " +
st.name());   // 0 NEW, 1 PAID ...
Status p = Status.valueOf("PAID");
// Status.valueOf("paid");  // IllegalArgumentException: names are case-
sensitive and exact
```

Fields, constructors and methods

```
public enum OrderStatus {
    NEW("N", "Order created"),
    PAID("P", "Payment received"),
    SHIPPED("S", "On the way"),
    CANCELLED("C", "Cancelled");

    private final String code;                  // fields should be final
    private final String description;

    OrderStatus(String code, String description) {          // implicitly
private
        this.code = code;
        this.description = description;
    }
    public String code()        { return code; }
    public String description() { return description; }

    private static final Map<String, OrderStatus> BY_CODE = new HashMap<>();
// lookup table
    static { for (OrderStatus s : values()) BY_CODE.put(s.code, s); }
// static block, NOT the constructor

    public static OrderStatus fromCode(String code) {
        OrderStatus s = BY_CODE.get(code);
        if (s == null) throw new IllegalArgumentException("unknown code: " +
```

```
code);
        return s;
    }
}
```

A constructor **cannot** read a static field of its own enum (the constants are created before the other statics are initialized), which is why the lookup map is filled in a `static` block after all constants exist.

Constant-specific bodies and abstract methods

Each constant can **override** methods, giving it its own behavior. The enum can declare an `abstract` method that every constant must implement.

```
public enum Operation {
    PLUS("+")  { @Override public double apply(double a, double b) { return a
+ b; } },
    MINUS("-") { @Override public double apply(double a, double b) { return a
- b; } },
    TIMES("*") { @Override public double apply(double a, double b) { return a
* b; } };

    private final String symbol;
    Operation(String symbol) { this.symbol = symbol; }
    public String symbol() { return symbol; }
    public abstract double apply(double a, double b);     // forces each
constant to define it
}

System.out.println(Operation.TIMES.apply(6, 7));          // 42.0
```

This is the **Strategy pattern** without a separate class hierarchy, and the compiler guarantees no constant forgets the method. A constant with a body is an anonymous subclass of the enum (so `getClass()` differs from the enum type; use `getDeclaringClass()`).

Enums implementing interfaces

```
interface Discount { double apply(double amount); }

enum Season implements Discount {
    FESTIVE { public double apply(double a) { return a * 0.8; } },
    REGULAR { public double apply(double a) { return a; } }
}
Discount d = Season.FESTIVE;                // an enum constant used through
the interface
```

State machines

Enums with behavior suit **small state machines**: each state knows its allowed transitions.

```
enum Light {
    RED    { Light next() { return GREEN; } },
    GREEN  { Light next() { return YELLOW; } },
    YELLOW { Light next() { return RED; } };
    abstract Light next();
}
Light l = Light.RED.next().next();          // YELLOW
```

Enums in switch

```
switch (status) {                                   // classic: case labels
use the SIMPLE name
    case NEW:  start(); break;
    case PAID: ship();  break;
    default:   break;
}

String label = switch (status) {                    // switch expression:
must be exhaustive
    case NEW       -> "waiting";
    case PAID      -> "ready";
    case SHIPPED   -> "moving";
    case CANCELLED -> "closed";                     // no default: the
compiler checks all constants are covered
};
```

If you later add `RETURNED`, the switch expression **stops compiling** until you handle it (a feature: omit `default` when you want that safety). A `null` selector throws `NullPointerException`.

EnumMap and EnumSet

Special collections for enum keys, backed by **arrays and bit vectors**; faster and more compact than `HashMap` and `HashSet`, and **iterated in ordinal order**.

```
EnumMap<Status, Integer> counts = new EnumMap<>(Status.class);
counts.merge(Status.PAID, 1, Integer::sum);

EnumSet<Status> open   = EnumSet.of(Status.NEW, Status.PAID);
EnumSet<Status> closed = EnumSet.complementOf(open);                  //
SHIPPED, CANCELLED
EnumSet<Day> weekdays  = EnumSet.range(Day.MON, Day.FRI);
EnumSet<Status> all    = EnumSet.allOf(Status.class);
EnumSet<Status> none   = EnumSet.noneOf(Status.class);
```

Prefer `EnumSet` over int bit flags, and `EnumMap` over `HashMap<Enum, ...>`.

The enum singleton

```
public enum AppConfig {
    INSTANCE;                                          // exactly one instance
    private final Properties props = load();
    public String get(String key) { return props.getProperty(key); }
    private Properties load() { return new Properties(); }
}
String v = AppConfig.INSTANCE.get("db.url");
```

It is thread-safe (class initialization is synchronized), safe against **serialization** (enums serialize by name and never create a second instance), and safe against **reflection** (the JVM refuses to instantiate enums reflectively). It is the singleton form recommended in *Effective Java*, though in Spring you normally let the container manage singleton beans.

Pitfalls and good practice

- **Never persist `ordinal()`.** Reordering or inserting a constant silently changes every stored value. Persist the **name** (`@Enumerated(EnumType.STRING)` in JPA) or an explicit **code** field.

- **`values()` returns a fresh array each call** (defensive copy). In a hot loop, cache it: `private static final Status[] ALL = values();`.
- **Prefer `==`** to `equals`; `Status.NEW == s` is null-safe and type-checked at compile time.
- **Keep enum fields `final`.** Mutable state in an enum constant is global mutable state.
- **Evolution:** adding a constant is source-compatible but can break exhaustive `switch`es in clients and data consumers (Jackson, databases) that do not know the new value; plan for unknown values (a default branch, `@JsonEnumDefaultValue`).
- **Do not use an enum when the set of values is open** or when each variant needs different data (use a sealed interface with records, 3.5.7).
- **JDK examples:** `DayOfWeek`, `Month`, `TimeUnit`, `RoundingMode`, `Thread.State`, `ChronoUnit`.

Interview traps

- *Can an enum extend a class? Implement an interface?* No to the first (it already extends `Enum`); yes to the second.
- *Can an enum have a constructor? Can it be `public`?* Yes it can have constructors, but they are implicitly `private`; `public` and `protected` are errors.
- *Can an enum have abstract methods?* Yes, if every constant provides a body.
- *`==` or `equals()` for enums?* `==`; `equals` is identity-based and final anyway.
- *What does `valueOf` do with an unknown name?* `IllegalArgumentException`.
- *Why is the enum a good singleton?* One instance guaranteed by the JVM, thread-safe initialization, and safe from serialization and reflection attacks.
- *Can you create an enum instance with `new` or reflection?* No.
- *What is the order of `Enum.compareTo`?* Declaration order (ordinal).

3.5.7 Sealed Classes and Pattern Matching: Closed Hierarchies

Normal inheritance is **open**: anyone can extend a non-final class. Enums are **closed** but every constant has the same shape. A **sealed** class or interface sits between them: a **closed set of subtypes, each with its own data**. Together with records and pattern matching it lets you model data the way functional languages do (**algebraic data types**).

Sealed classes and interfaces (Java 17)

A sealed type lists the **only** classes allowed to extend or implement it with `permits`.

```
public sealed abstract class Vehicle permits Car, Truck, Bike { }

public final class Car extends Vehicle { }                    // final: no
further subclasses
public sealed class Truck extends Vehicle permits Lorry { }   // sealed: its
own closed set
public final class Lorry extends Truck { }
public non-sealed class Bike extends Vehicle { }              // non-sealed:
open to anyone from here down

// public class Boat extends Vehicle { }     // COMPILE ERROR: Boat is not in
the permits list
```

Every permitted subclass must choose one of three modifiers:

Modifier	Meaning
`final`	No further subclasses
`sealed`	Further subclasses, but only the ones it lists with its own `permits`
`non-sealed`	Reopens the hierarchy: anyone may extend it

Rules

- Permitted subclasses must **directly** extend (or implement) the sealed type, and must be **accessible**.
- They must be in the **same module** (if the project is modular) or the **same package** (in the unnamed module).
- If all permitted subclasses are declared in the **same source file**, `permits` may be omitted; the compiler infers it.
- **Records and enums** are implicitly `final`, so they can be permitted subtypes without a modifier.
- A sealed type may be `abstract`, an interface, or a concrete class. Sealed **interfaces** are the most common form.
- Reflection: `Class.isSealed()` and `Class.getPermittedSubclasses()`.

Sealed interfaces with records: algebraic data types

```
sealed interface Expr permits Num, Add, Mul, Neg { }
record Num(double value)       implements Expr { }
record Add(Expr left, Expr right) implements Expr { }
record Mul(Expr left, Expr right) implements Expr { }
record Neg(Expr inner)         implements Expr { }

static double eval(Expr e) {
    return switch (e) {                                  // exhaustive: no
default needed
        case Num n               -> n.value();
        case Add(Expr l, Expr r) -> eval(l) + eval(r);   // record pattern:
takes the node apart
        case Mul(Expr l, Expr r) -> eval(l) * eval(r);
        case Neg(Expr x)         -> -eval(x);
    };
}

System.out.println(eval(new Add(new Num(2), new Mul(new Num(3), new
Num(4)))));   // 14.0
```

The compiler **knows every possible `Expr`**, so the `switch` needs no `default`. If you add `Div` to `permits`, `eval` stops compiling until you handle it: you cannot forget a case.

A domain example: payment results

```
sealed interface PaymentResult {
    record Approved(String txnId, double amount) implements PaymentResult { }
    record Declined(String reason)               implements PaymentResult { }
    record Pending(String reference)             implements PaymentResult { }
}

static String message(PaymentResult r) {
    return switch (r) {
        case PaymentResult.Approved a -> "paid " + a.amount() + " (" +
a.txnId() + ")";
        case PaymentResult.Declined d -> "declined: " + d.reason();
        case PaymentResult.Pending p  -> "waiting on " + p.reference();
    };
}
```

Because each variant carries **different data**, this is something an `enum` cannot express and a normal class hierarchy leaves open. Nested records of a sealed interface are implicitly permitted.

Pattern matching for `switch` (Java 21)

`switch` can test the **type** of its selector, bind a variable, and add conditions.

```
static String describe(Object o) {
    return switch (o) {
        case null                          -> "nothing";                  //
null is handled explicitly
        case Integer i when i > 100        -> "big int " + i;             //
guard with 'when'
        case Integer i                     -> "int " + i;
        case String s when s.isBlank()     -> "blank text";
        case String s                      -> "text of " + s.length();
        case int[] arr                     -> "int array of " + arr.length;
        case Add(Expr l, _)                -> "an addition";              //
unnamed pattern: ignore a component (Java 22+)
        default                            -> "something else";           //
required for open types like Object
    };
}
```

Rules

- **Order matters (dominance).** A case that matches everything an earlier case matches is a compile error:

```
switch (o) {
    case CharSequence cs -> "chars";
    case String s        -> "string";   // ERROR: dominated by CharSequence
above (every String is a CharSequence)
    default              -> "other";
}
```

Put **specific** cases first and general ones last; guarded cases before unguarded ones of the same type.

- **Exhaustiveness.** A switch expression, and a pattern switch statement, must cover every possible value. For a **sealed** type, covering all permitted subtypes is enough; for open types you need `default`.

- **`null`:** without `case null`, a `null` selector throws `NullPointerException`.
- **Do not add `default` to a switch over a sealed type unless you mean to.** It silences the exhaustiveness check, so a new subtype would go unnoticed at compile time.
- **Record patterns** can nest (`case Line(Point(var x1, var y1), Point p2)`), and `var` infers component types.
- **Unnamed variables and patterns** (`_`, final in Java 22) say "I do not use this".

Pattern matching with `instanceof`

```
if (o instanceof String s && s.length() > 3) {
System.out.println(s.toUpperCase()); }   // s in scope only where it is
definitely matched
if (!(o instanceof String s)) return;
// s is in scope after the return
System.out.println(s.length());
```

Evolution and compatibility

- **Adding a permitted subtype** is deliberately a breaking change: every non-exhaustive `switch` fails to compile on recompilation, which is the safety net.
- If a consumer is **not recompiled** (separate compilation), its old compiled `switch` throws `MatchException` when it meets the new subtype at runtime.
- Therefore use sealed hierarchies for **closed domain models** that you control (results, events, commands, ASTs, state machines), and open interfaces for **extension points** where outsiders must add implementations.

Choosing the right tool

Need	Use
Fixed constants, all the same shape	`enum`
Fixed variants, each with different data	**`sealed` interface + records**
Anyone may implement; behavior varies	Plain interface or abstract class
No subclasses at all	`final` class
Add new operations easily over a closed set of types	Sealed types + pattern `switch`

Need	Use
Add new types easily over a fixed set of operations	Interface with overridden methods (virtual dispatch)

Interview traps

- *What is a sealed class? What must permitted subclasses declare?* A class that restricts its direct subtypes with `permits`; each permitted subclass must be `final`, `sealed` or `non-sealed`.
- *Where must permitted subclasses live?* In the same module, or the same package for the unnamed module.
- *Why combine sealed types with records?* Records give immutable data carriers, sealed gives a closed set of them, and pattern matching takes them apart exhaustively.
- *What is dominance in a pattern switch?* A case already covered by an earlier case is unreachable and rejected by the compiler.
- *Do you need `default` for a sealed switch?* No; covering all permitted subtypes is exhaustive, and adding `default` loses the compile-time safety.
- *What happens when you add a new permitted subclass?* Switches that do not cover it fail to compile; precompiled callers throw `MatchException`.
- *Sealed vs enum?* Enums are fixed instances of one shape; sealed types are fixed **types** that can each carry different data.

Review Questions: Part 3.5

Attempt every question first. Part A is multiple choice (many are predict-the-output), Part B is short answer, Part C is coding and prediction, and Part D is senior level. All answers and explanations are consolidated in the next section.

Part A: Multiple Choice

Q1. What does the default `Object.toString()` return?

- (a) The field values
- (b) The class name only
- (c) The class name, `@`, and the hex form of the hash code

- (d) An empty string

Q2. What happens if a thread calls `obj.wait()` without holding `obj`'s monitor?

- (a) It waits normally
- (b) `IllegalMonitorStateException`
- (c) `InterruptedException`
- (d) Compile error

Q3. `x.equals(y)` is `true` but `y.equals(x)` is `false`. Which property of the `equals` contract is violated?

- (a) Reflexivity
- (b) Symmetry
- (c) Transitivity
- (d) Consistency

Q4. A class overrides `equals` but not `hashCode`. What is the most likely result?

```
Set<Money> set = new HashSet<>();
set.add(new Money(500));
System.out.println(set.contains(new Money(500)));
```

- (a) `true`
- (b) `false`
- (c) Compile error
- (d) `NullPointerException`

Q5. What happens?

```
class Point {
    int x, y;
    @Override public boolean equals(Point p) { return x == p.x && y == p.y; }
}
```

- (a) Compiles and overrides `equals`
- (b) Compile error: it does not override `equals(Object)`
- (c) Compiles, as an overload
- (d) Runtime error

Q6. With `Point.equals` using `instanceof Point` and `ColorPoint.equals` requiring `instanceof ColorPoint` plus matching color, what is printed?

```
Point p = new Point(1, 2);
ColorPoint cp = new ColorPoint(1, 2, "red");
System.out.println(p.equals(cp) + " " + cp.equals(p));
```

- (a) `true true`
- (b) `true false`
- (c) `false true`
- (d) `false false`

Q7. What is printed?

```
Set<Point2> set = new HashSet<>();   // Point2 is mutable; equals and
hashCode use x and y
Point2 p = new Point2(1, 2);
set.add(p);
p.x = 99;
System.out.println(set.contains(p));
```

- (a) `true`
- (b) `false`
- (c) `ConcurrentModificationException`
- (d) Compile error

Q8. Which collection operation uses only `equals()` and never `hashCode()`?

- (a) `HashMap.get`
- (b) `HashSet.contains`
- (c) `ArrayList.contains`
- (d) `LinkedHashMap.put`

Q9. What is `byAge.size()`?

```
TreeSet<Employee> byAge = new TreeSet<>
(Comparator.comparingInt(Employee::age));
byAge.add(new Employee("Asha", 30));
byAge.add(new Employee("Ravi", 30));
```

- (a) 2

- (b) 1
- (c) 0
- (d) `ClassCastException`

Q10. What are the two sizes printed?

```
BigDecimal a = new BigDecimal("1.0"), b = new BigDecimal("1.00");
System.out.println(new HashSet<>(List.of(a, b)).size());
System.out.println(new TreeSet<>(List.of(a, b)).size());
```

- (a) `1` and `1`
- (b) `2` and `2`
- (c) `2` and `1`
- (d) `1` and `2`

Q11. Which comparator is correct for `int` ages that may be large or negative?

- (a) `(a, b) -> a.age() - b.age()`
- (b) `(a, b) -> Integer.compare(a.age(), b.age())`
- (c) `(a, b) -> a.age() > b.age() ? 1 : 0`
- (d) `(a, b) -> a.age() < b.age()`

Q12. For employees Ravi (salary 70), Asha (50), Meera (70), what order results?

```
list.sort(Comparator.comparingInt(Employee::salary).reversed().thenComparing(
Employee::name));
```

- (a) Asha, Meera, Ravi
- (b) Meera, Ravi, Asha
- (c) Ravi, Meera, Asha
- (d) Asha, Ravi, Meera

Q13. `Collections.sort` on a `List<String>` is:

- (a) Stable (equal elements keep their relative order)
- (b) Unstable
- (c) Not supported for strings
- (d) Quadratic

Q14. Is this class immutable?

```
public final class Order {
    private final List<String> items;
    public Order(List<String> items) { this.items = items; }
    public List<String> getItems() { return items; }
}
```

- (a) Yes: the class is final and the field is final
- (b) No: the list is shared with the caller and exposed
- (c) Yes: there are no setters
- (d) Only if the list is empty

Q15. What is printed?

```
List<String> base = new ArrayList<>(List.of("a"));
List<String> view = Collections.unmodifiableList(base);
List<String> snap = List.copyOf(base);
base.add("b");
System.out.println(view + " " + snap);
```

- (a) `[a] [a]`
- (b) `[a, b] [a, b]`
- (c) `[a, b] [a]`
- (d) `[a] [a, b]`

Q16. Which line fails to compile?

```
final StringBuilder sb = new StringBuilder("a");   // L1
sb.append("b");                                    // L2
sb = new StringBuilder("c");                       // L3
```

- (a) L1
- (b) L2
- (c) L3
- (d) None

Q17. For `record Point(int x, int y) { }`, which accessor exists?

- (a) `getX()`
- (b) `x()`
- (c) `getx()`

- (d) None; you must write them

Q18. What happens?

```
record Person(String name) {
    private int age;
}
```

- (a) Compiles
- (b) Compile error: records cannot declare instance fields beyond their components
- (c) Compiles; `age` is `static`
- (d) Runtime error

Q19. What is printed?

```
record Scores(int[] values) { }
System.out.println(new Scores(new int[]{1}).equals(new Scores(new int[]
{1})));
```

- (a) `true`
- (b) `false`
- (c) Compile error
- (d) `NullPointerException`

Q20. In a compact constructor, `value = value.strip();` assigns:

- (a) The field directly
- (b) The constructor parameter; the field is assigned from it automatically afterwards
- (c) A new local variable
- (d) Nothing; it is an error

Q21. What happens?

```
enum Color { RED; public Color() { } }
```

- (a) Compiles
- (b) Compile error: an enum constructor cannot be `public`
- (c) Compiles with a warning
- (d) Runtime error

Q22. What does `Status.valueOf("paid")` do when the constant is `PAID`?

- (a) Returns `PAID`
- (b) Returns `null`
- (c) Throws `IllegalArgumentException`
- (d) Compile error

Q23. Which is the recommended way to compare enum constants?

- (a) `==`
- (b) `equals` only
- (c) `compareTo` only
- (d) `toString().equals(...)`

Q24. What happens?

```
class Base { }
enum E extends Base { A }
```

- (a) Compiles
- (b) Compile error: an enum cannot extend a class
- (c) Compiles with a warning
- (d) Runtime error

Q25. Which approach is risky for storing an enum in a database?

- (a) Storing `name()`
- (b) Storing an explicit code field
- (c) Storing `ordinal()`
- (d) Storing a lookup-table id

Q26. What happens?

```
sealed interface S permits A { }
final class A implements S { }
final class B implements S { }
```

- (a) Compiles
- (b) Compile error: `B` is not permitted
- (c) Compiles with a warning
- (d) Runtime error

Q27. A class named in a `permits` clause that declares none of `final`, `sealed` or `non-sealed` (and is not a record or enum) causes:

- (a) Nothing; it defaults to `final`
- (b) A compile error
- (c) A warning
- (d) It defaults to `non-sealed`

Q28. What happens?

```
switch (o) {
    case CharSequence cs -> System.out.println("chars");
    case String s        -> System.out.println("string");
    default              -> System.out.println("other");
}
```

- (a) Compiles; strings print `string`
- (b) Compile error: the `String` case is dominated by `CharSequence`
- (c) Compiles; strings print `chars`
- (d) Runtime error

Part B: Short Answer

Q29. List the methods of `Object`. Which should you usually override, and which should you never override or use?

Q30. Give guidelines for writing a good `toString()`, including security and JPA concerns.

Q31. State the rules for `wait`, `notify` and `notifyAll`, and explain why `wait` belongs in a loop.

Q32. State the five properties of the `equals` contract and a way each can be broken.

Q33. State the `hashCode` contract. What breaks if you override only `equals`? Walk through a `HashMap.get`.

Q34. Compare `instanceof` and `getClass()` in `equals`. Why is it impossible to extend an instantiable class, add a value component and keep `equals` correct, and what are the options?

Q35. Compare `Comparable` and `Comparator`. What does "consistent with equals" mean and what goes wrong without it?

Q36. List common comparator bugs and how to avoid them.

Q37. Give the recipe for an immutable class. Distinguish `final`, unmodifiable and immutable.

Q38. What does the compiler generate for a record? State its restrictions and the ways it is only shallowly immutable.

Q39. Explain why an enum is a full class. Cover fields, constructors, abstract methods, `EnumMap` / `EnumSet`, and why `ordinal()` should not be persisted.

Q40. What is a sealed class, what must permitted subclasses declare, and how do sealed types differ from enums and from `final` classes?

Part C: Coding and Prediction

C1. Write an immutable `Money` class (a `long` amount in paise and a `String` currency) with validation, `plus`, correct `equals`, `hashCode`, `toString` and `compareTo`.

C2. Repair this class so it is a correct value type used as a hash key:

```
class User {
    public String email;
    public List<String> roles = new ArrayList<>();
    @Override public boolean equals(Object o) { return o instanceof User u &&
u.email.equals(email); }
}
```

C3. Sort employees by department ascending, then salary descending, then name ascending, with employees whose `bonus` is `null` placed last when bonuses are equal on the first three keys.

C4. Make this class immutable:

```
class Roster {
    private List<String> names;
    private Date created;
    public List<String> getNames() { return names; }
    public void setNames(List<String> n) { names = n; }
    public Date getCreated() { return created; }
    public void setCreated(Date d) { created = d; }
}
```

C5. Write a record `Range(int lo, int hi)` that rejects `lo > hi`, with methods `contains(int)` and `overlaps(Range)`, and a record `Team(String name, List<String> members)` whose list cannot be changed from outside.

C6. Write an enum `Priority` with a numeric `weight`, a `fromWeight(int)` lookup, and an abstract method `slaHours()` implemented per constant. Count tasks per priority with an `EnumMap`.

C7. Model `PaymentResult` as a sealed interface with records `Approved(String txnId)`, `Declined(String reason)` and `Pending(String ref)`, write a method that handles every case with a `switch` expression, and say what happens when you add `Refunded`.

C8. Predict the output:

```
record P(int x, int y) { }
record Q(int[] v) { }
Set<P> set = new HashSet<>();
set.add(new P(1, 2));
set.add(new P(1, 2));
System.out.println(set.size());
System.out.println(new P(1, 2));
System.out.println(new Q(new int[]{1}).equals(new Q(new int[]{1})));
```

Part D: Senior Level

S1. Design `equals` and `hashCode` for a JPA entity whose id is generated by the database. Compare an id-based, a business-key and a constant-hash approach, and explain proxies and `getClass()`.

S2. Prove why a subclass that adds a value component cannot keep both symmetry and transitivity with its parent's `equals`, and list the ways to restructure the design.

S3. Explain how a poor `hashCode` affects `HashMap` performance, including treeification and what role `Comparable` plays. Why does `String` cache its hash?

S4. Describe how a broken `Comparator` leads to `IllegalArgumentException: Comparison method violates its general contract!`. Write a safe comparator for `Double` values that may be `NaN` or `null`.

S5. `TreeSet` and `TreeMap` use `compareTo` for identity. What design consequences follow, and how does this relate to `ConcurrentSkipListMap`?

S6. Immutability and the Java Memory Model: explain the final-field freeze guarantee, safe publication, and why an immutable class with a cached hash field is still safe. Design `Money` with `BigDecimal` and a currency.

S7. Record design trade-offs: invariants through the canonical constructor, evolving a record when a component is added, serialization and JSON, and why records are poor JPA entities.

S8. How do you evolve an enum that is persisted and exposed through an API? Compare storing ordinal, name and an explicit code, and plan for unknown values.

S9. A pricing rule engine has a handful of fixed rule kinds, each with different parameters, evaluated in many places. Compare an enum with abstract methods, a sealed interface with records, and a strategy interface. Choose and justify.

S10. Explain the expression problem and how it distinguishes virtual dispatch from sealed types with pattern `switch`. What are the risks of adding `default`, and of separate compilation (`MatchException`)?

S11. Compare `wait/notify` with `Condition` and `BlockingQueue`. How did virtual threads affect `synchronized` and `Object.wait`, and what changed in Java 24?

S12. Logging and `toString`: discuss sensitive data, cost, cycles and lazy evaluation of log arguments, with a safe pattern for entities.

Answers: Part 3.5

Part A: Multiple Choice

Q	Answer	Explanation
1	(c)	`ClassName@` plus the hex form of `hashCode()`.
2	(b)	The caller must own the monitor, otherwise `IllegalMonitorStateException`.
3	(b)	That is a symmetry violation.
4	(b)	Equal objects get different identity hash codes, so the lookup usually searches another bucket and misses.

Q	Answer	Explanation
5	(b)	The parameter type is `Point`, not `Object`, so it overloads; `@Override` turns that into a compile error.
6	(b) `true false`	`p.equals(cp)` passes the `instanceof Point` test; `cp.equals(p)` fails because `p` is not a `ColorPoint`. Symmetry is broken.
7	(b)	The hash changed after insertion, so the lookup goes to a different bucket.
8	(c)	`ArrayList` is a list and has no hash table. The other three hash first.
9	(b) 1	The comparator returns 0 for the same age, so `TreeSet` treats Ravi as a duplicate of Asha.
10	(c)	`HashSet` uses `equals` (scale differs, so 2); `TreeSet` uses `compareTo` (numerically equal, so 1).
11	(b)	Subtraction can overflow, option (c) is not antisymmetric, and (d) returns a `boolean`.
12	(b)	Salary descending puts both 70s first; ties break by name ascending: Meera before Ravi; then Asha.
13	(a)	Object sorting uses TimSort, which is stable.
14	(b)	The list comes from the caller and is exposed, so both can mutate the order.
15	(c) `[a, b] [a]`	The unmodifiable view follows the original; `List.copyOf` made an independent snapshot.
16	(c)	A `final` reference cannot be reassigned, but the object it points to can change (L2 is fine).
17	(b)	Record accessors are named after the component.
18	(b)	Records cannot declare instance fields other than their components.
19	(b)	Array components are compared by reference.

Q	Answer	Explanation
20	(b)	In a compact constructor the names refer to the parameters; the fields are assigned from them at the end.
21	(b)	Enum constructors are implicitly `private`; `public` or `protected` is an error.
22	(c)	`valueOf` needs the exact name, otherwise `IllegalArgumentException`.
23	(a)	`==` is null-safe, type-checked, and correct because each constant is a singleton.
24	(b)	An enum already extends `java.lang.Enum`.
25	(c)	Reordering or inserting constants changes ordinals and corrupts stored data.
26	(b)	`B` is not in the `permits` list.
27	(b)	A permitted subclass must be `final`, `sealed` or `non-sealed` (records and enums are implicitly final).
28	(b)	Every `String` is a `CharSequence`, so the second case can never be reached (dominance).

Part B: Short Answer (model answers)

Q29. `getClass`, `hashCode`, `equals`, `toString`, `clone`, `finalize`, `wait` (three overloads), `notify`, `notifyAll`. Usually override `toString`, and `equals` with `hashCode` for value types. `getClass`, `wait`, `notify` and `notifyAll` are `final`. Avoid overriding `clone` (use copy constructors) and never override or rely on `finalize` (deprecated for removal).

Q30. Print the significant state in a compact, consistent format; never include secrets or personal data; do not trigger lazy loading or remote calls; avoid cycles in bidirectional relationships (print an id for the other side); use `Objects.toString` for nullable parts; use `Arrays.toString` for arrays; for JPA entities print the id and business key only. Records and enums generate a `toString`, but a record prints all components, so override it when one of them is sensitive.

Q31. The caller must own the object's monitor (`synchronized`). `wait()` releases the monitor and waits; `notify()` wakes one waiting thread and `notifyAll()` all of them, and a woken thread must re-acquire the lock before continuing. `wait` goes in a loop (`while (!condition) wait();`) because of spurious wake-ups and because several conditions can share one monitor. Prefer `notifyAll` unless you can prove one waiter is enough, and prefer `java.util.concurrent` in new code.

Q32. Reflexive (`x.equals(x)` ; broken by a faulty field comparison), **symmetric** (broken by comparing with other types, or subclass overrides), **transitive** (broken by a subclass that adds a field), **consistent** (broken by volatile data such as time or remote state) and **non-null** (`x.equals(null)` must be `false` ; broken by not checking and throwing instead).

Q33. Within a run, `hashCode` is stable while the `equals` data is unchanged, and **equal objects must have equal hash codes**; unequal objects may share a hash code. If only `equals` is overridden, equal objects keep different identity hashes. `HashMap.get(key)` computes the hash, selects the bucket from it, then uses `equals` on that bucket's entries; with mismatched hashes an equal key is searched for in the wrong bucket, so lookups miss and sets accept duplicates.

Q34. `instanceof` lets a subclass instance equal a parent instance (substitutable) but breaks symmetry or transitivity when a subclass adds state; `getClass()` keeps symmetry and transitivity but violates Liskov (an anonymous or proxy subclass is not "equal") and fails with framework proxies. Take `Point(1,2)` and `ColorPoint(1,2,red)` : if `ColorPoint.equals(Point)` returns false, symmetry breaks; if it ignores the color, two different `ColorPoint` s each equal the same `Point` but not each other, so transitivity breaks. Options: make the classes `final` , use records, use composition (`ColorPoint` has a `Point`), or make the parent abstract.

Q35. `Comparable` defines one natural order inside the class (`compareTo`); a `Comparator` defines any number of external orders (`compare`), works for classes you do not own, and is a lambda target. "Consistent with equals" means `compareTo == 0` exactly when `equals` is true. Without it, `HashSet` (equals) and `TreeSet` (compareTo) disagree, and `TreeSet` / `TreeMap` silently drop elements that compare as 0 (the `BigDecimal` and partial-key comparator examples).

Q36. Subtraction overflow (use `Integer.compare`); comparing doubles with `<` (use `Double.compare` ; `NaN`); returning only 1 or 0 (not antisymmetric); comparators that ignore `null` ; inconsistent or random comparators (TimSort throws

`IllegalArgumentException`); missing tie-breakers, so distinct objects compare as 0 in a `TreeSet`; mutating a key while it is inside a tree.

Q37. No mutators; `final` class; `final` and `private` fields; defensive copies of mutable inputs in the constructor and of mutable outputs in accessors (or return immutable types); no `this` escape. `final` fixes only the reference; an **unmodifiable** wrapper is a read-only view of a collection that others can still change; **immutable** means nobody can change it (`List.of`, `List.copyOf`, `String`). A class is deeply immutable only if everything reachable from it is.

Q38. A record gets private final fields, a canonical constructor, accessors named after the components, `equals`, `hashCode` and `toString`; it is `final` and extends `Record`. It cannot extend a class, declare extra instance fields, or have setters; it can implement interfaces, have static members, extra constructors that delegate, and compact constructors for validation. It is shallowly immutable: a mutable component object (a list, an array, a `Date`) can still change unless copied defensively, and arrays compare by reference.

Q39. An enum is a class whose constants are its only instances (`public static final`), extending `java.lang.Enum`, with a private constructor, fields, methods, interface implementations and even constant-specific bodies with an abstract method each constant must implement (the Strategy pattern). `EnumMap` and `EnumSet` are array and bit-vector based, ordered by ordinal, and faster than hash collections. `ordinal()` must not be persisted because reordering or inserting a constant changes every stored value; store the name or an explicit code.

Q40. A sealed class or interface restricts its direct subtypes to those named in `permits` (same module or package); each must be `final`, `sealed` or `non-sealed`. An `enum` is a closed set of **instances** of one shape; a sealed type is a closed set of **types**, each possibly with different data (records). A `final` class allows no subclasses at all; a sealed class allows a known few, and lets pattern `switch` be exhaustive without `default`.

Part C: Coding and Prediction (model solutions)

C1. Money

```
public final class Money implements Comparable<Money> {
    private static final Comparator<Money> ORDER =
Comparator.comparing(Money::currency).thenComparingLong(Money::paise);
    private final long paise;
    private final String currency;
```

```
    public Money(long paise, String currency) {
        if (paise < 0) throw new IllegalArgumentException("negative amount: "
+ paise);
        this.paise = paise;
        this.currency = Objects.requireNonNull(currency, "currency");
    }
    public long paise()      { return paise; }
    public String currency() { return currency; }

    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new
IllegalArgumentException("currency mismatch");
        return new Money(Math.addExact(paise, other.paise), currency);
// exact: fails instead of overflowing
    }
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Money m)) return false;
        return paise == m.paise && currency.equals(m.currency);
    }
    @Override public int hashCode() { return Objects.hash(paise, currency); }
    @Override public String toString() { return currency + " " + paise / 100
+ "." + String.format("%02d", paise % 100); }
    @Override public int compareTo(Money o) { return ORDER.compare(this, o);
}   // total order, consistent with equals
}
```

C2. A correct value type and hash key

```
public final class User {
    private final String email;
    private final List<String> roles;

    public User(String email, List<String> roles) {
        this.email = Objects.requireNonNull(email).strip().toLowerCase();
// normalize once
        this.roles = List.copyOf(roles);
// defensive copy, immutable
    }
    public String email()        { return email; }
    public List<String> roles()  { return roles; }

    @Override public boolean equals(Object o) { return o instanceof User u &&
email.equals(u.email); }   // identity = email
    @Override public int hashCode() { return email.hashCode(); }
// same field
}
```

The original had public mutable fields, no `hashCode`, and could be mutated after being placed in a set. Roles are deliberately outside `equals`: two `User` objects for the same email are the same user.

C3. Multi-key comparator

```
Comparator<Employee> cmp = Comparator.comparing(Employee::department)
        .thenComparing(Employee::salary, Comparator.reverseOrder())
        .thenComparing(Employee::name)
        .thenComparing(Employee::bonus,
Comparator.nullsLast(Comparator.naturalOrder()));
staff.sort(cmp);
```

C4. Immutable `Roster`

```
public final class Roster {
    private final List<String> names;
    private final Instant created;                          // immutable
replacement for Date

    public Roster(List<String> names, Instant created) {
        this.names = List.copyOf(names);
        this.created = Objects.requireNonNull(created);
    }
    public List<String> names() { return names; }
    public Instant created()    { return created; }
    public Roster withNames(List<String> n) { return new Roster(n, created);
}   // wither instead of a setter
}
```

If the public API must keep `Date`, copy it in the constructor and in the getter (`new Date(d.getTime())`).

C5. Records

```
public record Range(int lo, int hi) {
    public Range {
        if (lo > hi) throw new IllegalArgumentException("lo " + lo + " > hi "
+ hi);
    }
    public boolean contains(int v)      { return v >= lo && v <= hi; }
    public boolean overlaps(Range o)    { return lo <= o.hi && o.lo <= hi; }
}

public record Team(String name, List<String> members) {
    public Team {
        Objects.requireNonNull(name, "name");
        members = List.copyOf(members);                     // outside code
can no longer change the roster
    }
}
```

C6. Enum with behavior

```
enum Priority {
    LOW(1)    { @Override int slaHours() { return 72; } },
    MEDIUM(2) { @Override int slaHours() { return 24; } },
    HIGH(3)   { @Override int slaHours() { return 4;  } };
```

```
    private final int weight;
    Priority(int weight) { this.weight = weight; }
    int weight() { return weight; }
    abstract int slaHours();

    private static final Map<Integer, Priority> BY_WEIGHT = new HashMap<>();
    static { for (Priority p : values()) BY_WEIGHT.put(p.weight, p); }
    static Priority fromWeight(int w) {
        Priority p = BY_WEIGHT.get(w);
        if (p == null) throw new IllegalArgumentException("no priority with
weight " + w);
        return p;
    }
}

EnumMap<Priority, Integer> counts = new EnumMap<>(Priority.class);
for (Task t : tasks) counts.merge(t.priority(), 1, Integer::sum);
```

C7. Sealed result

```
sealed interface PaymentResult {
    record Approved(String txnId) implements PaymentResult { }
    record Declined(String reason) implements PaymentResult { }
    record Pending(String ref)     implements PaymentResult { }
}

static String message(PaymentResult r) {
    return switch (r) {
        case PaymentResult.Approved a -> "approved " + a.txnId();
        case PaymentResult.Declined d -> "declined: " + d.reason();
        case PaymentResult.Pending p  -> "pending " + p.ref();
    };
}
```

Adding `record Refunded(String txnId) implements PaymentResult` makes `message` fail to compile ("the switch expression does not cover all possible input values") until a `Refunded` case is added. That is the benefit of omitting `default`.

C8. Output

```
1
P[x=1, y=2]
false
```

The two `P(1, 2)` records are equal with equal hash codes, so the set holds one; `toString` is generated; and `Q`'s array component is compared by reference, so two records holding equal-looking arrays are not equal.

Part D: Senior Level (model answers)

S1. A database-generated `id` is `null` until `persist`, so an `id`-based `hashCode` changes when the entity is saved, losing it from a `HashSet`. Options: (1) **Business key** (an immutable natural identifier such as email or ISBN) in both methods: stable and meaningful, if such a key exists. (2) **`id`-based `equals` plus a constant `hashCode`** (for example the class's hash): correct before and after persistence, with `equals` returning `false` while `id == null` except for the same instance; hashing cost is O(n) per entity type in large sets. (3) **Assign the id up front** (UUID created in the constructor, or application-assigned) so `id`-based methods work from the start. Never include lazy collections or relationships (lazy loading and recursion). Because Hibernate returns **proxy subclasses**, compare with `instanceof` (or `Hibernate.getClass`) rather than `getClass()`, and read fields through getters, since a proxy's own fields are not populated. Lombok's `@Data` and `@EqualsAndHashCode` on entities cause exactly these problems.

S2. Let `P = Point(1,2)`, `R = ColorPoint(1,2,red)` and `B = ColorPoint(1,2,blue)`. Suppose `Point.equals` uses `instanceof Point` so `P.equals(R)` and `P.equals(B)` are `true` (a colored point is a point). *Case 1:* `ColorPoint.equals(Point)` returns `false`, so `R.equals(P)` is `false` while `P.equals(R)` is `true`: **symmetry broken**. *Case 2:* `ColorPoint.equals(Point)` ignores color, so `R.equals(P)` and `P.equals(B)` are `true`, but `R.equals(B)` is `false` (colors differ): **transitivity broken**. With `getClass()` both properties hold, but `P.equals(anonymousSubclassOfPoint)` is false for equal state, violating **Liskov**. Hence no instantiable class can be extended with a new value component without breaking the contract or substitutability. Restructure: **composition** (`ColorPoint` has a `Point` and a color), **final classes or records**, an **abstract parent** with no instances of its own, or `canEqual`-style protocols in languages that use them.

S3. `HashMap` spreads the hash (`h ^ (h >>> 16)`) and masks it with the table size. With a poor hash (many collisions, or a constant), entries pile up in a few buckets. Chains grow, and once a bin has 8 or more nodes and the table has at least 64 buckets, it is converted to

a **red-black tree**: lookups drop from O(n) to O(log n), and keys that are `Comparable` (of the same class) are ordered by `compareTo`, so the tree can be searched efficiently; otherwise the tree falls back to tie-breaking by class name and identity hash and may have to search both subtrees. Treeification also defeats hash-flooding attacks. `String` caches its hash in a field because strings are immutable and are the most common keys, so the hash is computed once (since Java 13 there is also a flag, so strings whose hash is 0 are not recomputed each time).

S4. TimSort merges sorted runs and relies on the comparator being a **total order**. When inconsistent results are observed (a before b, b before c, c before a), the merge detects the impossibility and throws `IllegalArgumentException: Comparison method violates its general contract!`, but only for some data sets, because it is detected only if the sort actually exercises the bad comparisons. Causes: returning only `1` or `0`, `NaN` with `<` / `>`, overflow subtraction, `null`, random or time-dependent keys, mutable keys. A safe comparator:

```
Comparator<Double> safe = Comparator.nullsLast(Comparator.
<Double>naturalOrder());   // null last; NaN greatest; -0.0 before 0.0
// primitive doubles: Double.compare(a, b)  (total order)
```

S5. `TreeSet` and `TreeMap` treat two keys as the **same** when the comparison returns 0, whatever `equals` says. So `add` rejects an element that compares equal, `contains`, `get` and `remove` find by comparison, and a comparator that checks only some fields makes the structure a set of **equivalence classes** (useful on purpose: a case-insensitive `TreeMap` with `String.CASE_INSENSITIVE_ORDER`). Consequences: use immutable keys; make the comparator consistent with `equals` unless you want the grouping; null keys are rejected under natural ordering; mixing incomparable types throws `ClassCastException`; and `Set.equals` between a tree set and a hash set can be asymmetric. `ConcurrentSkipListMap` and `ConcurrentSkipListSet` follow exactly the same rules, with lock-free thread safety.

S6. The JLS (17.5) says that when a constructor ends, the object's **final fields are frozen**: any thread that obtains a reference, even through a data race, sees those fields (and objects reachable only through them) as they were at the end of construction, as long as `this` did not escape. Non-final state needs **safe publication** (volatile, a lock, a concurrent collection, a static initializer). A cached `hash` field (like `String`'s) is safe because it is an `int` (writes are atomic), is derived deterministically from final state, and a thread that sees the stale `0` just recomputes the same value; a `long` could tear and a non-deterministic cache would not be benign. A sound `Money`:

```
public final class Money {
    private final BigDecimal amount;           // scale normalized to the
currency's fraction digits
    private final Currency currency;           // java.util.Currency:
immutable
    private Money(BigDecimal amount, Currency c) { this.amount = amount;
this.currency = c; }
    public static Money of(String amount, String code) {
        Currency c = Currency.getInstance(code);
        BigDecimal a = new
BigDecimal(amount).setScale(c.getDefaultFractionDigits(),
RoundingMode.UNNECESSARY);
        return new Money(a, c);                // equals and hashCode are
then safe on amount + currency
    }
    public Money plus(Money o) { requireSame(o); return new
Money(amount.add(o.amount), currency); }
    private void requireSame(Money o) { if (!currency.equals(o.currency))
throw new IllegalArgumentException(); }
    @Override public boolean equals(Object o) { return o instanceof Money m
&& amount.equals(m.amount) && currency.equals(m.currency); }
    @Override public int hashCode() { return Objects.hash(amount, currency);
}
}
```

Normalizing the scale up front makes `BigDecimal.equals` safe (otherwise `1.0` and `1.00` differ). Alternatively store minor units in a `long`.

S7. The **canonical constructor** is the single place for invariants and every instance, including one rebuilt by deserialization or Jackson, passes through it. A record's components are its public API, so **adding a component** changes the constructor, accessors, `equals`, `hashCode`, `toString` and the serialized shape: source, binary and wire compatibility all break. Mitigate by keeping the old canonical form as an extra constructor that delegates with a default, using builders for large records, and versioning DTOs. Jackson 2.12+ reads records through the canonical constructor (missing properties arrive as `null` / `0`, so validate). Java serialization of records also goes through the canonical constructor, so it cannot bypass your checks. Records are poor **JPA entities** (a no-arg constructor, mutable fields and proxies are required), but fine as `@Embeddable` (Hibernate 6.2+), projections and DTOs. Also: no hidden representation, no inheritance, shallow immutability, and array components need care.

S8. Store a stable, explicit **code** (or the name), never the ordinal: ordinal changes on reorder or insertion, name changes on rename. A code field decouples storage from the Java identifier, so you can rename constants safely. In JPA use `@Enumerated(EnumType.STRING)` or an `AttributeConverter` that maps the code. Plan for **unknown values**: a database or message may hold a value your older code does not know, so `fromCode` should return an `UNKNOWN` constant or an `Optional`, and Jackson can use `READ_UNKNOWN_ENUM_VALUES_AS_NULL` or `@JsonEnumDefaultValue`. Adding a constant is source-compatible but can break clients with exhaustive `switch`es, so document it and version the API; deprecate rather than delete. If values change without redeployment (reference data edited by operators), an enum is the wrong tool: use a lookup table with a foreign key.

S9. Rule kinds are fixed but each has **different parameters** (`PercentOff(pct)`, `BulkDiscount(minQty, amount)`, `BuyXGetY(x, y)`), and many operations apply to them (price, explain, validate, serialize). **Sealed interface plus records** fits: records carry the per-instance data, and each operation is a pattern `switch` that the compiler checks for exhaustiveness, so a new rule kind shows every place to update, and a new operation is just another function. An **enum with abstract methods** cannot work because enum constants are singletons: they cannot hold different parameters per rule instance. A **strategy interface** is right when the rule set is **open** (plugins, rules loaded from configuration) or when the behavior needs injected services; then use one bean per strategy. Choose sealed plus records for the closed domain model, and add a strategy only if open extension is a real requirement.

S10. The **expression problem**: with **virtual dispatch** (an interface and one class per type), adding a **type** is easy but adding an **operation** means editing every class (or using Visitor); with a **sealed hierarchy and pattern `switch`**, adding an **operation** is easy, and adding a **type** is flagged by the compiler at every non-exhaustive switch. Risks: a `default` branch silences the exhaustiveness check, so a new subtype falls into `default` unnoticed; and under **separate compilation**, a client compiled against the old hierarchy throws `MatchException` at runtime when the library adds a subtype. So treat adding a permitted subtype as a breaking change at module boundaries, keep switches close to the type, and use `default` only when you intend open behavior.

S11. `wait/notify` gives one wait set per object and no timeouts on the lock itself; `Lock` with `Condition` allows several conditions per lock (`notFull`, `notEmpty`), interruptible and timed acquisition, and fairness; `BlockingQueue` packages the whole producer-consumer protocol. All of them need the condition re-check loop. With **virtual threads** (Java 21), blocking in a `synchronized` block or `Object.wait()` **pinned** the virtual thread

to its carrier thread, hurting scalability, while `ReentrantLock` and `Condition` did not pin. **Java 24 (JEP 491)** lets virtual threads unmount while blocked on a monitor or in `Object.wait()` (with a few exceptions such as native frames), so `synchronized` is no longer a scalability problem for them. Still prefer the `java.util.concurrent` classes for new code.

S12. `toString` output flows to logs, so exclude secrets and personal data (mask them or wrap them in a type whose `toString` prints `***`; a record prints every component, so override it or wrap the sensitive component). Building the string costs time and memory: `log.debug("user " + user)` calls `toString` even when debug is off, while SLF4J's `log.debug("user {}", user)` or a supplier calls it only when enabled. Avoid lazy loading, remote calls, and unbounded collections inside `toString`, and break cycles. A safe pattern for entities is to print only the id and business key, and log DTOs or small records built for the purpose; Lombok's `@ToString.Exclude` helps; structured logging with the MDC carries correlation ids instead of dumping objects.

End of Part 3.5. The pass-by-value material planned for this part was covered earlier in 3.1.6. Next is **Part 3.6: the `java.lang` package in depth** (the String class internals, `StringBuilder` and `StringBuffer`, wrapper classes, and other important `java.lang` classes) with a capstone on design principles and exercises. Tell me when to start, or flag anything here you want expanded first.